

Task 2

Integrantes

- Sergio Orellana 221122
- Rodrigo Mansilla 22611
- Ricardo Chuy 221007

Utilice PyTorch o TensorFlow/Keras (a su elección). Debe escribir el código desde cero o basarse en las documentaciones oficiales. Ejecute sus experimentos en Google Colab, Kaggle Notebooks o en su GPU local. Sientanse libres de hacer uso de IA de forma educada, es decir, entendiendo realmente que lo que esté haciendo sea realmente lo que necesitan y que sobretodo lo entiendan.

1. Descargue el dataset, divídalo en Entrenamiento (70%), Validación (15%) y Prueba (15%). Implemente Data Augmentation (rotaciones, flips, recortes) vital para evitar el sobreajuste en imágenes agrícolas.
2. Cargue los siguientes modelos pre-entrenados en ImageNet y congele sus capas base, reemplazando solo el cabezal de clasificación para nuestras 8 clases de mango:
 - a. ResNet (Puede usar ResNet50).
 - b. Inception (Puede usar InceptionV3).
 - c. MobileNet (Puede usar MobileNetV2 o V3-Small/Large).
3. Entrene los 3 modelos utilizando la misma función de pérdida (Cross-Entropy) y optimizador (ej. Adam) por un máximo de 15 a 20 épocas (o use Early Stopping).
4. Registre las métricas correspondientes para cada modelo:
 - a. Accuracy (Exactitud) en el conjunto de prueba.
 - b. F1-Score (Macro) en el conjunto de prueba.
 - c. Tamaño del modelo final (en Megabytes) al guardarlo en disco (.pth o .h5).
 - d. Tiempo de Inferencia: Cuántos milisegundos (ms) tarda en predecir una sola imagen (promedio sobre 100 imágenes).

```
In [7]: import os
import time
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
from torchvision import transforms, datasets, models
from torch.utils.data import DataLoader, random_split
from sklearn.metrics import f1_score, accuracy_score
```

```
import numpy as np
import kagglehub
```

```
In [8]: # para gpu en colab
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(f"Usando dispositivo: {device}")

# 1. Descargar Dataset
print("Descargando dataset desde Kaggle...")
dataset_path = kagglehub.dataset_download("aryashah2k/mango-leaf-disease-dataset")
print("Ruta del dataset:", dataset_path)

# Transformaciones con Data Augmentatio
# Para entrenamiento, agregamos rotaciones, flips y recortes
train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

test_transforms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

dataset_completo = datasets.ImageFolder(root=dataset_path)
clases = dataset_completo.classes
print(f"Clases a usar ({len(clases)}):", clases)

total_imagenes = len(dataset_completo)
train_size = int(0.70 * total_imagenes)
val_size = int(0.15 * total_imagenes)
test_size = total_imagenes - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(
    dataset_completo, [train_size, val_size, test_size],
    generator=torch.Generator().manual_seed(42)
)

# clase de pytorch necesaria
class DatasetWrapper(torch.utils.data.Dataset):
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform
    def __getitem__(self, index):
        x, y = self.subset[index]
        if self.transform:
            x = self.transform(x)
        return x, y
    def __len__(self):
        return len(self.subset)
```

```

train_dataset = DatasetWrapper(train_dataset, transform=train_transforms)
val_dataset = DatasetWrapper(val_dataset, transform=test_transforms)
test_dataset = DatasetWrapper(test_dataset, transform=test_transforms)

print()
print(f"Distribución - Entrenamiento: {len(train_dataset)}, Validación: {len(val_da

BATCH_SIZE = 32
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

```

Usando dispositivo: cuda:0

Descargando dataset desde Kaggle...

Using Colab cache for faster access to the 'mango-leaf-disease-dataset' dataset.

Ruta del dataset: /kaggle/input/mango-leaf-disease-dataset

Clases a usr (8): ['Anthracnose', 'Bacterial Canker', 'Cutting Weevil', 'Die Back', 'Gall Midge', 'Healthy', 'Powdery Mildew', 'Sooty Mould']

Distribución - Entrenamiento: 2800, Validación: 600, Prueba: 600

Resnet

```

In [9]: print("Primer modelo: RESNET")
modelo_resnet = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)

# aca es donde se congelan las capas base
for param in modelo_resnet.parameters():
    param.requires_grad = False

# modificar ultima capa para que clasifique manguitos
num_fts = modelo_resnet.fc.in_features
modelo_resnet.fc = nn.Linear(num_fts, len(clases))

modelo_resnet = modelo_resnet.to(device)
criterio = nn.CrossEntropyLoss()
optimizador = optim.Adam(modelo_resnet.fc.parameters(), lr=0.001)

#entrenamiento con 15 epocas
NUM_EPOCAS = 15

print("Comenzando entrenamiento_____")
tiempo_inicio_entrenamiento = time.time()

for epoca in range(NUM_EPOCAS):
    modelo_resnet.train()
    running_loss = 0.0

    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizador.zero_grad()
        outputs = modelo_resnet(inputs)

```

```

        loss = criterio(outputs, labels)

        loss.backward()
        optimizador.step()

        running_loss += loss.item() * inputs.size(0)

    epoch_loss = running_loss / len(train_dataset)
    print(f'Época {epoca+1}/{NUM_EPOCAS} - Pérdida: {epoch_loss:.4f}')

tiempo_fin_entrenamiento = time.time()
print(f"Entrenamiento completado en {(tiempo_fin_entrenamiento - tiempo_inicio_entre

```

Primer modelo: RESNET

Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth

100%|██████████| 97.8M/97.8M [00:00<00:00, 194MB/s]

Comenzando entrenamiento_____

Época 1/15 - Pérdida (Loss): 1.1406

Época 2/15 - Pérdida (Loss): 0.4878

Época 3/15 - Pérdida (Loss): 0.3295

Época 4/15 - Pérdida (Loss): 0.2611

Época 5/15 - Pérdida (Loss): 0.2326

Época 6/15 - Pérdida (Loss): 0.1930

Época 7/15 - Pérdida (Loss): 0.1722

Época 8/15 - Pérdida (Loss): 0.1644

Época 9/15 - Pérdida (Loss): 0.1489

Época 10/15 - Pérdida (Loss): 0.1389

Época 11/15 - Pérdida (Loss): 0.1301

Época 12/15 - Pérdida (Loss): 0.1241

Época 13/15 - Pérdida (Loss): 0.1220

Época 14/15 - Pérdida (Loss): 0.1161

Época 15/15 - Pérdida (Loss): 0.1093

Entrenamiento completado en 4.70 minutos.

Guardar el modelo

```

In [11]: ruta_modelo = "resnet_mango.pth"
         torch.save(modelo_resnet.state_dict(), ruta_modelo)
         tamano_mb = os.path.getsize(ruta_modelo) / (1024 * 1024)

```

Evaluación de Resnet

```

In [15]: modelo_resnet.eval()
         todas_las_predicciones = []
         todas_las_etiquetas = []
         tiempos_inferencia = []

         print("Calculando Accuracy y F1-Score_____")
         with torch.no_grad():
             for inputs, labels in test_loader:
                 inputs, labels = inputs.to(device), labels.to(device)
                 outputs = modelo_resnet(inputs)

```

```

_, predicciones = torch.max(outputs, 1)
todas_las_predicciones.extend(predicciones.cpu().numpy())
todas_las_etiquetas.extend(labels.cpu().numpy())

# metricas
exactitud = accuracy_score(todas_las_etiquetas, todas_las_predicciones)
f1_macro = f1_score(todas_las_etiquetas, todas_las_predicciones, average='macro')

# para probar con exactamente 100 imagenes, lo que se hace seria un batch loader pp
# predecir de 1 en 1 y medir tiempo, para luego hacer un promedio
tiempos_inferencia = []
imagenes_procesadas = 0
test_loader_single = DataLoader(test_dataset, batch_size=1, shuffle=True)

print("Probando las 100 inferencias_____")
with torch.no_grad():
    for inputs, _ in test_loader_single:
        if imagenes_procesadas >= 100:
            break

        inputs = inputs.to(device)

        if torch.cuda.is_available():
            torch.cuda.synchronize()

        start_inf = time.time()
        _ = modelo_resnet(inputs)

        if torch.cuda.is_available():
            torch.cuda.synchronize()

        end_inf = time.time()

        tiempo_en_ms = (end_inf - start_inf) * 1000
        tiempos_inferencia.append(tiempo_en_ms)
        imagenes_procesadas += 1
tiempo_inferencia_promedio = np.mean(tiempos_inferencia)

print(f"Accuracy: {exactitud * 100:.2f}%")
print(f"F1-Score Macro: {f1_macro:.4f}")
print(f"Tamaño del modelo guardado: {tamano_mb:.2f} MB")
print(f"Tiempo de inferencia promedio: {tiempo_inferencia_promedio:.2f} ms")

```

Calculando Accuracy y F1-Score_____.
 Probando las 100 inferencias_____
 Accuracy (Prueba): 99.00%
 F1-Score Macro (Prueba): 0.9900
 Tamaño del modelo guardado: 90.04 MB
 Tiempo de inferencia promedio: 6.54 ms

InceptionV3

```

In [16]: # en este caso, aunque ya tengamos los datos separados al principio del jupyter
# es necesario tener en cuenta algunos cambios, incluso en las transformaciones
# principalmente por las dimensiones de imagen

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(f"Usando dispositivo: {device}")

# descarga para tener las imagenes frescas
dataset_path = kagglehub.dataset_download("aryashah2k/mango-leaf-disease-dataset")

# un cambio necesario es para las dimensiones de las imagenes
train_transforms_inc = transforms.Compose([
    transforms.RandomResizedCrop(299),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

test_transforms_inc = transforms.Compose([
    transforms.Resize(320),
    transforms.CenterCrop(299), # Tamaño para Inception
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

dataset_completo = datasets.ImageFolder(root=dataset_path)
clases = dataset_completo.classes

total_imagenes = len(dataset_completo)
train_size = int(0.70 * total_imagenes)
val_size = int(0.15 * total_imagenes)
test_size = total_imagenes - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(
    dataset_completo, [train_size, val_size, test_size],
    generator=torch.Generator().manual_seed(42)
)

class DatasetWrapper(torch.utils.data.Dataset):
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform
    def __getitem__(self, index):
        x, y = self.subset[index]
        if self.transform:
            x = self.transform(x)
        return x, y
    def __len__(self):
        return len(self.subset)

train_dataset_inc = DatasetWrapper(train_dataset, transform=train_transforms_inc)
val_dataset_inc = DatasetWrapper(val_dataset, transform=test_transforms_inc)
test_dataset_inc = DatasetWrapper(test_dataset, transform=test_transforms_inc)

```

```

BATCH_SIZE = 32
train_loader_inc = DataLoader(train_dataset_inc, batch_size=BATCH_SIZE, shuffle=True)
val_loader_inc = DataLoader(val_dataset_inc, batch_size=BATCH_SIZE, shuffle=False)
test_loader_inc = DataLoader(test_dataset_inc, batch_size=BATCH_SIZE, shuffle=False)

```

Usando dispositivo: cuda:0

Using Colab cache for faster access to the 'mango-leaf-disease-dataset' dataset.

```

In [17]: modelo_inception = models.inception_v3(weights=models.Inception_V3_Weights.DEFAULT)

# congelar base
for param in modelo_inception.parameters():
    param.requires_grad = False

#
num_ftrs_aux = modelo_inception.AuxLogits.fc.in_features
modelo_inception.AuxLogits.fc = nn.Linear(num_ftrs_aux, len(clases))

num_ftrs = modelo_inception.fc.in_features
modelo_inception.fc = nn.Linear(num_ftrs, len(clases))

modelo_inception = modelo_inception.to(device)

criterio = nn.CrossEntropyLoss()
# otra diferencia importante es que es necesario optimizar las capas
# usamos adma ya que es uno de los mas comunes
optimizador = optim.Adam([
    {'params': modelo_inception.fc.parameters()},
    {'params': modelo_inception.AuxLogits.fc.parameters()}
], lr=0.001)

NUM_EPOCAS = 15
print("Comenzando entrenamiento de InceptionV3___")
tiempo_inicio = time.time()

for epoca in range(NUM_EPOCAS):
    modelo_inception.train()
    running_loss = 0.0

    for inputs, labels in train_loader_inc:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizador.zero_grad()

        # otro punto dfierente es que inception devuelve las salidas principales y
        outputs, aux_outputs = modelo_inception(inputs)

        loss1 = criterio(outputs, labels)
        loss2 = criterio(aux_outputs, labels)
        loss = loss1 + 0.4 * loss2

        loss.backward()
        optimizador.step()

        running_loss += loss.item() * inputs.size(0)

```

```

epoch_loss = running_loss / len(train_dataset_inc)
print(f'Época {epoca+1}/{NUM_EPOCAS} - Pérdida (Loss): {epoch_loss:.4f}')

tiempo_fin = time.time()
print(f"Entrenamiento InceptionV3 completado en {(tiempo_fin - tiempo_inicio)/60:.2}

```

Downloading: "https://download.pytorch.org/models/inception_v3_google-0cc3c7bd.pth" to /root/.cache/torch/hub/checkpoints/inception_v3_google-0cc3c7bd.pth

100%|██████████| 104M/104M [00:00<00:00, 194MB/s]

Comenzando entrenamiento de InceptionV3__

Época 1/15 - Pérdida (Loss): 1.6930

Época 2/15 - Pérdida (Loss): 0.8602

Época 3/15 - Pérdida (Loss): 0.6492

Época 4/15 - Pérdida (Loss): 0.5714

Época 5/15 - Pérdida (Loss): 0.5319

Época 6/15 - Pérdida (Loss): 0.4649

Época 7/15 - Pérdida (Loss): 0.4776

Época 8/15 - Pérdida (Loss): 0.4345

Época 9/15 - Pérdida (Loss): 0.4306

Época 10/15 - Pérdida (Loss): 0.4702

Época 11/15 - Pérdida (Loss): 0.4253

Época 12/15 - Pérdida (Loss): 0.3880

Época 13/15 - Pérdida (Loss): 0.4038

Época 14/15 - Pérdida (Loss): 0.3731

Época 15/15 - Pérdida (Loss): 0.3831

Entrenamiento InceptionV3 completado en 6.54 minutos.

Guardar el modelo

```

In [18]: ruta_modelo_inc = "inception_mango.pth"
torch.save(modelo_inception.state_dict(), ruta_modelo_inc)
tamano_mb_inc = os.path.getsize(ruta_modelo_inc) / (1024 * 1024)

```

Metricas

```

In [20]: modelo_inception.eval()
todas_las_predicciones = []
todas_las_etiquetas = []

print("Calculando Accuracy y F1-Score__")
with torch.no_grad():
    for inputs, labels in test_loader_inc:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = modelo_inception(inputs)

        _, predicciones = torch.max(outputs, 1)
        todas_las_predicciones.extend(predicciones.cpu().numpy())
        todas_las_etiquetas.extend(labels.cpu().numpy())

exactitud_inc = accuracy_score(todas_las_etiquetas, todas_las_predicciones)
f1_macro_inc = f1_score(todas_las_etiquetas, todas_las_predicciones, average='macro')

print("Midiendo tiempo de inferencia para 100 imágenes__")

```



```

tiempos_inferencia = []
imagenes_procesadas = 0

test_loader_single = DataLoader(test_dataset_inc, batch_size=1, shuffle=True)

if torch.cuda.is_available():
    dummy_input = torch.randn(1, 3, 299, 299).to(device)
    for _ in range(10): _ = modelo_inception(dummy_input)

with torch.no_grad():
    for inputs, _ in test_loader_single:
        if imagenes_procesadas >= 100: break

        inputs = inputs.to(device)

        if torch.cuda.is_available(): torch.cuda.synchronize()
        start_inf = time.time()

        _ = modelo_inception(inputs)

        if torch.cuda.is_available(): torch.cuda.synchronize()
        end_inf = time.time()

        tiempos_inferencia.append((end_inf - start_inf) * 1000)
        imagenes_procesadas += 1

tiempo_inferencia_promedio_inc = np.mean(tiempos_inferencia)

print(f"Accuracy: {exactitud_inc * 100:.2f}%")
print(f"F1-Score Macro: {f1_macro_inc:.4f}")
print(f"Tamaño del modelo guardado: {tamano_mb_inc:.2f} MB")
print(f"Tiempo de inferencia promedio (sobre 100 imgs): {tiempo_inferencia_promedio

```

Calculando Accuracy y F1-Score____
Midiendo tiempo de inferencia para 100 imágenes____
Accuracy: 96.83%
F1-Score Macro: 0.9677
Tamaño del modelo guardado: 93.28 MB
Tiempo de inferencia promedio (sobre 100 imgs): 11.68 ms

MobileNet

```

In [21]: device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(f"Usando dispositivo: {device}")

dataset_path = kagglehub.dataset_download("aryashah2k/mango-leaf-disease-dataset")

train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

```

```

test_transforms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

dataset_completo = datasets.ImageFolder(root=dataset_path)
classes = dataset_completo.classes

total_imagenes = len(dataset_completo)
train_size = int(0.70 * total_imagenes)
val_size = int(0.15 * total_imagenes)
test_size = total_imagenes - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(
    dataset_completo, [train_size, val_size, test_size],
    generator=torch.Generator().manual_seed(42)
)

train_dataset_mob = DatasetWrapper(train_dataset, transform=train_transforms)
val_dataset_mob = DatasetWrapper(val_dataset, transform=test_transforms)
test_dataset_mob = DatasetWrapper(test_dataset, transform=test_transforms)

BATCH_SIZE = 32
train_loader_mob = DataLoader(train_dataset_mob, batch_size=BATCH_SIZE, shuffle=True)
val_loader_mob = DataLoader(val_dataset_mob, batch_size=BATCH_SIZE, shuffle=False)
test_loader_mob = DataLoader(test_dataset_mob, batch_size=BATCH_SIZE, shuffle=False)

```

Usando dispositivo: cuda:0

Using Colab cache for faster access to the 'mango-leaf-disease-dataset' dataset.

```

In [22]: print("Mobilenet v2")
modelo_mobilenet = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)

for param in modelo_mobilenet.parameters():
    param.requires_grad = False

# en mobilnet el clasificador es un bloque secunciall
num_fts = modelo_mobilenet.classifier[1].in_features
modelo_mobilenet.classifier[1] = nn.Linear(num_fts, len(classes))

modelo_mobilenet = modelo_mobilenet.to(device)

criterio = nn.CrossEntropyLoss()
optimizador = optim.Adam(modelo_mobilenet.classifier.parameters(), lr=0.001)

NUM_EPOCAS = 15
print("Comenzando entrenamiento de MobileNetV2...")
tiempo_inicio = time.time()

for epoca in range(NUM_EPOCAS):
    modelo_mobilenet.train()
    running_loss = 0.0

```

```

for inputs, labels in train_loader_mob:
    inputs, labels = inputs.to(device), labels.to(device)

    optimizador.zero_grad()
    outputs = modelo_mobilenet(inputs)
    loss = criterio(outputs, labels)

    loss.backward()
    optimizador.step()

    running_loss += loss.item() * inputs.size(0)

epoch_loss = running_loss / len(train_dataset_mob)
print(f'Época {epoca+1}/{NUM_EPOCAS} - Pérdida (Loss): {epoch_loss:.4f}')

tiempo_fin = time.time()
print(f"Entrenamiento MobileNetV2 completado en {(tiempo_fin - tiempo_inicio)/60:.2

```

MobileNet v2

Downloading: "https://download.pytorch.org/models/mobilenet_v2-7ebf99e0.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v2-7ebf99e0.pth

100%|██████████| 13.6M/13.6M [00:00<00:00, 63.4MB/s]

Comenzando entrenamiento de MobileNetV2...

Época 1/15 - Pérdida (Loss): 1.1003

Época 2/15 - Pérdida (Loss): 0.5022

Época 3/15 - Pérdida (Loss): 0.3564

Época 4/15 - Pérdida (Loss): 0.2976

Época 5/15 - Pérdida (Loss): 0.2595

Época 6/15 - Pérdida (Loss): 0.2342

Época 7/15 - Pérdida (Loss): 0.2295

Época 8/15 - Pérdida (Loss): 0.2170

Época 9/15 - Pérdida (Loss): 0.1991

Época 10/15 - Pérdida (Loss): 0.1864

Época 11/15 - Pérdida (Loss): 0.1906

Época 12/15 - Pérdida (Loss): 0.1725

Época 13/15 - Pérdida (Loss): 0.1880

Época 14/15 - Pérdida (Loss): 0.1800

Época 15/15 - Pérdida (Loss): 0.1576

Entrenamiento MobileNetV2 completado en 3.30 minutos.

Guardar modelo

```

In [23]: ruta_modelo_mob = "mobilenet_mango.pth"
torch.save(modelo_mobilenet.state_dict(), ruta_modelo_mob)
tamano_mb_mob = os.path.getsize(ruta_modelo_mob) / (1024 * 1024)

```

Metricas

```

In [25]: modelo_mobilenet.eval()
todas_las_predicciones = []
todas_las_etiquetas = []

print("Calculando Accuracy y F1-Score...")

```

```

with torch.no_grad():
    for inputs, labels in test_loader_mob:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = modelo_mobilenet(inputs)

        _, predicciones = torch.max(outputs, 1)
        todas_las_predicciones.extend(predicciones.cpu().numpy())
        todas_las_etiquetas.extend(labels.cpu().numpy())

exactitud_mob = accuracy_score(todas_las_etiquetas, todas_las_predicciones)
f1_macro_mob = f1_score(todas_las_etiquetas, todas_las_predicciones, average='macro')

print("Midiendo tiempo de inferencia para 100 imágenes...")
tiempos_inferencia = []
imagenes_procesadas = 0

test_loader_single = DataLoader(test_dataset_mob, batch_size=1, shuffle=True)

if torch.cuda.is_available():
    dummy_input = torch.randn(1, 3, 224, 224).to(device)
    for _ in range(10): _ = modelo_mobilenet(dummy_input)

with torch.no_grad():
    for inputs, _ in test_loader_single:
        if imagenes_procesadas >= 100: break

        inputs = inputs.to(device)

        if torch.cuda.is_available(): torch.cuda.synchronize()
        start_inf = time.time()

        _ = modelo_mobilenet(inputs)

        if torch.cuda.is_available(): torch.cuda.synchronize()
        end_inf = time.time()

        tiempos_inferencia.append((end_inf - start_inf) * 1000)
        imagenes_procesadas += 1

tiempo_inferencia_promedio_mob = np.mean(tiempos_inferencia)

print(f"Accuracy (Prueba): {exactitud_mob * 100:.2f}%")
print(f"F1-Score Macro (Prueba): {f1_macro_mob:.4f}")
print(f"Tamaño del modelo guardado: {tamano_mb_mob:.2f} MB")
print(f"Tiempo de inferencia promedio (sobre 100 imgs): {tiempo_inferencia_promedio} ms")

```

Calculando Accuracy y F1-Score...

Midiendo tiempo de inferencia para 100 imágenes...

Accuracy (Prueba): 99.00%

F1-Score Macro (Prueba): 0.9901

Tamaño del modelo guardado: 8.76 MB

Tiempo de inferencia promedio (sobre 100 imgs): 5.37 ms